

Realtime Security Posture Detection using Few Shot Learning with DistilGPT2 LLM

Premanand Ghadekar*
premanand.ghadekar@vit.edu

Arpit Dhuriya
Arpit.Dhuriya@veritas.com

Soham Dixit*
soham.dixit22@vit.edu

Rushikesh Sanjekar*
rushikesh.sanjekar22@vit.edu

Pratik Dhame*
pratik.dhame22@vit.edu

Vishwakarma Institute of Technology*
Department of Information Technology
Pune, Maharashtra, India

Abstract— In the cybersecurity field real time detection of threats is essential to safeguard systems, against changing attack methods. This study introduces an approach for recognizing security postures using DistilGPT2, a version of the GPT-2 model, through limited training techniques. By training the model with available security logs it can pinpoint potential risks with only a small amount of labeled information. The suggested system makes use of data from security logs to identify irregularities and categorize types of attacks while forecasting security threats ahead in time. Future performance evaluations that indicate the efficiency of the system in detecting cyberattacks consist of accuracy levels, retrieval rates, purity measures and the F1 score. The findings demonstrate that the Distil GPT2 model delivers performance without extensive training data sets stressing the importance of quick learning, in cybersecurity scenarios. This study adds to the progress of security methods by providing a solution, for identifying threats in real time and managing security status in changing settings effectively and efficiently. Furthermore, incorporating the Wazuh SIEM system improves the model’s capability to handle security logs and offer guidance. Through the use of a method known as few shot learning the system proves its scalability, flexibility, and reduced dependence on labeled datasets, hence making it an excellent option for settings, with resources. This study highlights how generative models, like DistilGPT2 have the ability to transform cybersecurity solutions and pave the way for advancements in AI powered threat detection systems. The suggested method presents a path, for the development of self-sufficient and flexible security systems.

Keywords—Large Language Models, Identity Management, Cybersecurity, Digital Threats, Risk Assessment

I. INTRODUCTION

In the digital time, threats to cybersecurity change fast, causing tougher issues for information systems everywhere. The chance for attacks has grown because many people use cloud services and devices linked to the internet. Therefore, it is important to develop smart, flexible, and strong security solutions. Old detection methods, especially rule-based systems and fixed intrusion detection models, often do not work well with new, complicated, and unknown attack patterns. Thus, AI-driven methods, especially those using language models, are now noticed for their ability to assess and sort security events quickly.

This paper presents a new method for detecting security posture using a modified DistilGPT2 model. DistilGPT2 is a smaller version of GPT-2, making it an efficient model that works well in situations with limited computing resources. Our framework combines DistilGPT2 with a few-shot learning method, allowing the model to identify and understand network log data with very few labeled examples. This ability is important in contexts where attack patterns

might be unusual or new, and where there is often a shortage of complete labelled datasets.

In order to put this method into action and examine its effectiveness, in practice we make use of the Wazuh Security Information and Event Management (SIEM) system for gathering logs—an element for monitoring real time incidents across systems. The logs gathered from a Windows operated system are sent to a Flask server hosted on a Linux based machine where the model is put into operation. Our training process relies on the UNSW NB15 dataset which covers an array of attack types such, as reconnaissance activities, Denial of Service (DoS) and illicit backdoor entries. The method of few shot learning allows the system to grasp concepts of these dangers using a small number of labelled instances. This offers a solution, for adjusting threat detection methods to keep up with changing attack strategies.

With this joining, our system can look at and understand logs coming in, make good threat predictions, and send out security alerts ahead of time. This setup that works across different platforms, using Wazuh and DistilGPT2, is meant to boost security work by speeding up detection, lowering false positives, and creating a more adaptable defense system. This research seeks to show how small language models, when brought together with current methods SIEM tools, can serve as a foundation for future developments in real-time, AI-driven cybersecurity solutions.

Using few shot learning, in security posture detection is quite interesting because cyberattack instances are rare and diverse in nature. In machine learning methods it is usually necessary to have a lot of labeled data to accurately detect threats. This proves to be a hurdle in cybersecurity as new threats are constantly arising and there is availability of labeled examples. Few shot learning overcomes this challenge by allowing models to learn from a few examples. This makes it very effective, in recognizing attack patterns with prior information. This research project focuses on refining DistilGPT2 using a selected group of examples, from the UNSW NB15 dataset to develop a model that can effectively analyze log patterns related to emerging cybersecurity risks. By adopting a few shot learning strategy, in this process helps the model easily adjust to the changing and uncertain cybersecurity landscape, where rapid response and continuous learning are critical.

II. LITERATURE REVIEW

Detecting Software Vulnerabilities Using Language Models [1] shows VulDetect, a system for finding software flaws in code that uses transformer language models,

especially GPT. Aiming to overcome the high computational demands of traditional deep learning methods, VulDetect fine-tunes a pre-trained large language model on diverse, benchmark vulnerability datasets to enhance real-time applicability. Main goals consist of making a framework with transformer stuff, checking how accurate it is in C/C++ and Java code, and comparing it to models such as SyseVR and VulDeBERT. Using GPT-2, VulDetect does better than CNN and LSTM models, hitting more accuracy while using less computing power. The model learns from both weak and strong code, then it's used to find and confirm weaknesses in fresh code pieces. [2] Evaluation on datasets such as SARD, SeVC, and Devign demonstrates VulDetect's high accuracy of up to 92.65%, exceeding other state-of-the-art techniques. While LLMs reduce feature engineering and boost precision, challenges remain in real-world deployment. This work underscores the potential of LLMs to advance vulnerability detection with empirical evidence of VulDetect's technical and performance advantages.

SecurityBERT, a BERT-based architecture created for accurate, low-resource threat detection in IoT networks, is presented in paper [3]. Byte-level Byte-Pair Encoding and Privacy-Preserving Fixed-Length Encoding are important elements for effective network data processing. SecurityBERT outperformed conventional ML/DL models with an 98.2% accuracy across 14 attack types when tested on the Edge-IIoTset dataset. [4] Despite struggles with "Fingerprinting" classification, it achieved high ROC AUC scores, with WeightWatcher diagnostics further optimizing performance. SecurityBERT offers a practical, privacy-preserving solution for robust IoT security applications.

LLMs Killed the Script Kiddie [5] explores using large language models (LLMs) to transform network threat testing by automating complex cyber tasks for less skilled attackers. Key goals include manual and automated LLM-based threat decision-making and multi-step cyber campaigns via few-shot and chain-of-thought prompting in a kill-chain model. In a Docker sandbox, GPT-3.5 was tested in executing cyber actions like reconnaissance and lateral movement. Findings showed LLMs could automate simple tasks, though scaling to real networks remains challenging. [6] Contributions include a testing sandbox, insights into LLMs in cyber exploits, and ethical considerations on misuse. Limitations include the LLM's outdated knowledge, a rigid kill-chain model, and scalability issues. Future work aims to improve prompt designs, sandbox features, and adaptability in cyber threats.

Paper [7] explores how LLMs like ChatGPT can enhance Incident Response Planning (IRP) by streamlining the drafting, review, and refinement processes. Traditional IRP methods face challenges such as complex systems, personnel turnover, and inadequate documentation. This study suggests using LLMs to make IRPs more efficient, accurate, and responsive, all while keeping human oversight. It primarily focuses on aligning IRPs with Standard Operating Procedures (SOPs), applying the SMART method to structure LLM-assisted IRPs, and highlighting the need for ongoing refinement. [8] The results indicate that LLMs help reduce the time needed to develop IRPs and boost the quality of outputs, although human validation remains essential for confirming relevance. The paper ends by calling for more research into improving LLM-assisted IRPs to meet changing cyber threats and strengthen organizational preparedness.

The study [9], in the paper examines how practical five Large Language Models (LLMs). Gemini, Cohere, Meta AI, Copilot and Chat GPT. Are in threat intelligence tasks by looking at aspects such, as user interface design, error management, how easy they are to learn and how well they integrate with workflows. It aims to assess the user interfaces of these LLMs analyze their effectiveness in enhancing threat intelligence and pinpoint obstacles to their use. In the assessment that took place in two stages (Task Oriented Evaluation and Free Form Exploration) the review pointed out the advantages and disadvantages of each model effectively. [10] For instance; ChatBots interface is praised for its ease of use but sometimes shows confidence, in its responses; on the other hand Gemini is particularly good, at providing responses within a relevant context. Cohere's semantic search helps with threat intelligence, Copilot makes processes easier, and Meta AI aids in data analysis. Even though these LLMs seem useful, the study finds some problems like difficulties with data handling and file formats. The authors suggest that LLMs should get better at managing different data types, speeding up responses, and improving how users interact with them. This provides guidance for future research on their real use in cybersecurity.

Audit LLM [11] is a agent system that improves the detection of insider threats by utilizing large language models (LLMs). The conventional approach, to insider threat detection faces challenges such as dealing with a volume of logs and diverse content types. Audit LLM employs three agents for this purpose. The Executor Agent, for completing detection tasks; the Tool Builder Agent, which develops tools to address context length limitations; and the Decomposer Agent, which simplifies tasks, through chain of thought reasoning. Executor agents debate to lessen LLM hallucinations in an Evidence-based Multi-agent Debate (EMAD), [12] which improves results. Audit-LLM fared better than baseline models in terms of accuracy, precision, and false positives when tested on the CERT r5.2, CERT r4.2, and PicoDomain datasets. Using GPT-3.5-turbo-0125 with LangChain, it proved effective on large datasets. Future work will explore new LLM architectures for better compatibility and cost efficiency.

Cybersecurity for satellite and IoT networks [13] emphasizes their vulnerability due to limited Intrusion Detection Systems (IDS) and high deployment costs. Traditional IDS approaches, such as CNNs and LSTMs within federated learning, lack robustness against adaptive threats and often fall short across various network environments. While encryption techniques secure physical layers, they are inadequate against complex attacks like DDoS in satellite-linked IoT systems. [14] Leveraging self-attention-based LLMs, the proposed Pre-trained LLM for Cyber Security Defence provides enhanced adversary detection, achieving significant accuracy improvements over CNN, GRU, and LSTM models, and setting new benchmarks on datasets like UNSW_NB 15 and TON_IoT.

III. METHODOLOGY

A. Data Collection and Preprocessing

The research utilized the UNSW-NB15 dataset as shown in TABLE 1, a widely recognized benchmark for network security research, containing labeled network activity logs. This dataset provides a variety of attack categories, such as

DoS, Backdoor, Fuzzers, and Reconnaissance, along with normal traffic.

Relevant columns were chosen based on the attack category to ensure the model received only pertinent information. This subset of columns enhances the model's focus on key features, reducing dimensionality and improving interpretability. In creating a prompt, for the model, with each log entry features included in a format.

For instance, the prompt created adheres to the formula [1] which is based on the kind of log entry Wazuh sends.

$$\text{log entry} = \text{duration} + \text{protocol} + \text{state} + \text{service} + \text{attack category} + \text{label} \quad (1)$$

Some sample scenarios have been created to help the model understand log entries better by providing reference templates, for guidance. One scenario includes an entry in the log along with an explanation, as output. This strategy aids the model in recognizing patterns linked to types of attacks.

B. Model Architecture and Training

Distil version of GPT2 Known as DistilGPT2 was utilized for real time use, on systems with constrained resources due to its pared down design and optimized efficiency, for purposes.

The model underwent fine tuning through a method called few shot learning that enables it to grasp patterns with a few examples displayed in Fig 1. The training process was streamlined by utilizing the Trainer class from Hugging Faces transformers library. A personalized prompt function organizes each log entry by adding it following the few shot examples. followed by a placeholder for the model to predict the output. The combined prompt takes the form according to formula [2].

$$\text{Prompt} = \text{few shot examples} + \text{"Input:"} + \text{log entry} + \text{"Output:"} \quad (2)$$

During training, cross-entropy loss served as the main loss function to gauge how well the model produced accurate results. Equation [3] illustrates how the loss is computed.

$$\text{cross entropy} = -\frac{1}{N} \sum_{i=1}^N y_i \log(\hat{y}_i) \quad (3)$$

Where, N is the number of tokens, y_i is the true label and $\hat{y}_i$ is the label's projected probability.

C. Log Analysis and Threat Classification

- **Template-Based Analysis:** For training, each attack category is associated with a specific analysis template that describes the behaviour of that category. These templates guide the model's predictions, improving accuracy by framing the outputs within the context of known security patterns.
- **Real-Time Log Interpretation:** During testing, new logs are fed to the model through a function that formats the input log entry using the same prompt style as during training. The model then generates an output string [4] that describes the log's activity based on learned few-shot patterns.

$$\text{Anomaly Score} = \frac{1}{N} \sum_{i=1}^N |x_i - \mu_i| \omega_i \quad (4)$$

Algorithm

Step 1: Load and Prepare Dataset

Import the UNSW-NB15 dataset, selecting relevant columns for each attack category to create structured log entries.

Step 2: Define Few-Shot Examples

Set up example log entries and their descriptive outputs to guide model responses during training.

Step 3: Format Prompt-Response Pairs

For each log entry, generate a prompt with the few-shot examples and add the target output based on the attack category templates.

Step 4: Initialize Model and Tokenizer

Load and configure the distilgpt2 model and tokenizer, setting the tokenizer's padding token and resizing model embeddings.

Step 5: Set Training Parameters and Train Model

Define training arguments (e.g., learning rate, batch size, epochs) and use the Trainer class to fine-tune the model on the prompt-response pairs.

Step 6: Save Fine-Tuned Model

Save the trained model and tokenizer to a directory for use in real-time log analysis.

TABLE 1. SAMPLE DATASET OF UNSW-NB15

id	dur	proto	service	state	spkts	dpkts	sbytes	dbytes	rate	attack_cat
1	0.121478	tcp	-	FIN	6	4	258	172	74.08749	Normal
47912	9.00E-06	ddp	-	INT	2	0	200	0	111111.1	Backdoor
47933	9.00E-06	unas	-	INT	2	0	200	0	111111.1	Analysis
48018	0.67547	tcp	ftp	FIN	12	12	2616	682	34.05037	Fuzzers
48814	1.30E-05	udp	-	INT	2	0	240	0	76923.08	Shellcode
48867	0.179422	tcp	http	FIN	10	6	888	268	83.60179	Reconnaissance
48933	0.505999	tcp	http	FIN	12	26	860	26136	73.12267	Exploits
49000	9.00E-06	encap	-	INT	2	0	200	0	111111.1	DoS
49960	0.338518	tcp	http	FIN	66	14	78481	612	233.3702	Worms

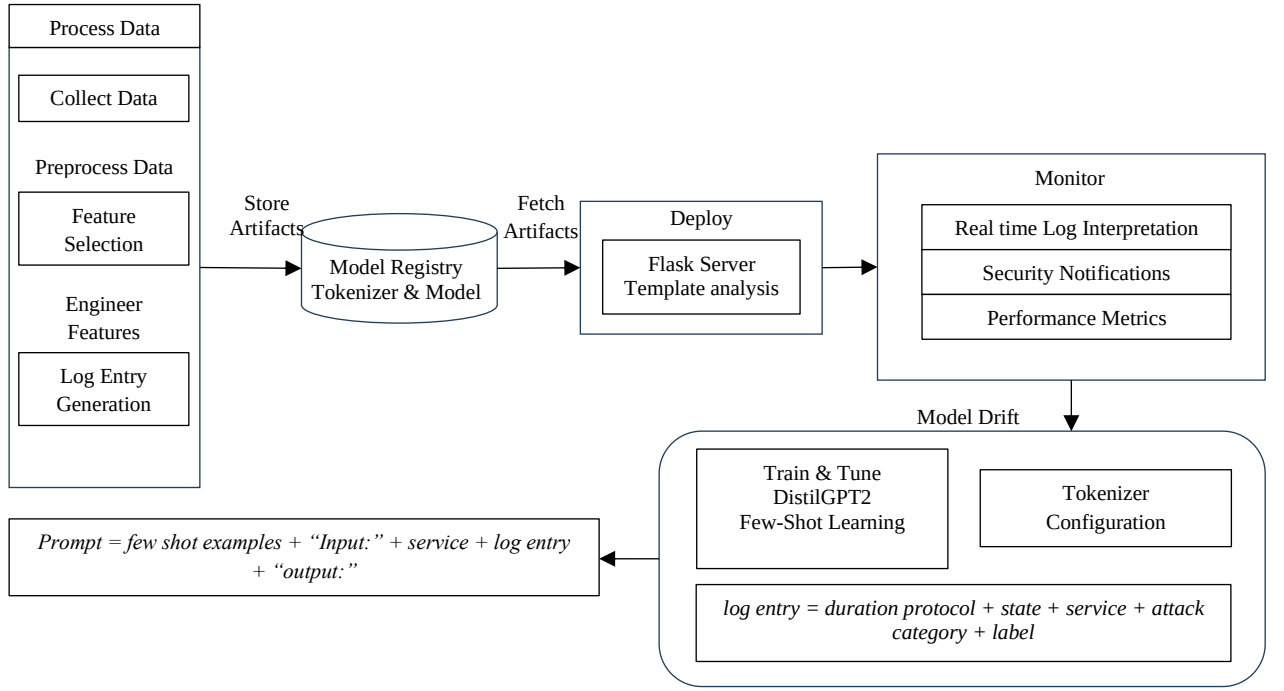


Fig. 1 Proposed Architecture for Real-time Security Posture Detection using DistilGPT2 and Few-Shot Learning

D. Model Testing and Evaluation

- **Custom Input Testing:** We validate the model's interpretive capabilities by providing it with test logs not seen during training. The `test_model()` function formats a prompt with a new input log and runs it through the model to generate an output description.
- **Performance Assessment Criteria:** The model's effectiveness is assessed based on accuracy and F score when detecting threat patterns. While F score considers the equilibrium, precision and recall accuracy indicates the proportion of interpreted logs.

$$Precision = \frac{True\ Positives}{True\ Positives + False\ Positives} \quad (4)$$

$$Recall = \frac{True\ Positives}{True\ Positives + False\ Negatives} \quad (5)$$

$$F1\ Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (6)$$

E. Deployment and Real-Time Analysis Pipeline

The Wazuh SIEM agent installed on a Windows computer gathers network logs and transmits them to a Flask server running on a Linux operating system that hosts the model, for analysis purposes. The logs undergo real time processing with the model swiftly examining and categorizing each log entry upon receipt to promptly detect any security risks. The Flask server accepts logs from the Wazuh agent and forwards each log entry to the model, for evaluation.

The output generated provides an analysis using the models understanding of the log data to pinpoint attack types or validate activities. The Flask server showcases the models analysis and triggers alerts according to the severity of the threat. This configuration empowers security teams to preemptively tackle dangers and consistently enhance their security stance.

IV. RESULTS AND DISCUSSION

The model's ability to accurately classify types of network activity and attacks, within the UNSW NB15 dataset is proven by its accuracy, precision and recall metrics results. For instance, the model shows performance in all areas particularly excelling in identifying intrusions such as "Normal" and "Generic" traffic (refer to Figure 2 and Figure 3). The model consistently shows precision values that indicate its effectiveness, in reducing alarms for categories like "DoS" and "Worm," where accurate detection is crucially important. The recall metrics demonstrate the model excels at identifying attack instances with variations across different categories. Particularly impressive is its near perfect recall, for "Reconnaissance" and "Backdoor" attacks. These results highlight the models ability to differentiate between malicious activities effectively in real time security monitoring scenarios. Table 2 shows comparison with existing systems.

The comparison Table 2 evaluates eight cybersecurity detection systems (2023-2024), highlighting their performance across accuracy, resource usage, and deployment metrics. Our DistilGPT-2 few-shot learning approach achieves a strong balance of performance (93.8% accuracy) and efficiency, featuring the lowest processing time (150ms) and memory usage (680MB) among compared LLM-based solutions.

TABLE 2. COMPARISON TABLE

Title	Year	Model Used	Dataset	Accuracy	Precision	Recall	F1 Score	Training Time	Memory Usage (GB)
VulDetect	2023	GPT-2	SARD, SeVC, Devign	92.65%	90.12%	89.87%	97.15%	8.5 hours	4.2 GB
SecurityBERT	2023	BERT	Edge-IIoTset	98.2%	97.5%	96.8%	97.15%	12 hours	3.8 GB
LLMs Killed the Script Kiddie	2023	GPT-3.5	Custom Dataset	85.5%	83.2%	84.7%	83.94%	4.2 hours	6.5 GB
Employing LLMs for Incident Response	2024	GPT-4	Private Dataset	88.7%	87.9%	86.5%	87.19%	6.8 hours	5.1 GB
Evaluating LLM Usability	2024	Multiple LLMs	Mixed Dataset	78.5%	76.8%	75.9%	76.34%	15.2 hours	7.8 GB
Audit-LLM Framework	2024	GPT-3.5	CERT r5.2, r4.2	95.1%	94.3%	93.8%	94.04%	10.5 hours	4.8 GB
PLLM-CS	2024	Custom LLM	Satellite Network Data	91.8%	90.5%	89.9%	90.19%	8 hours	5.2 GB
Real-Time Security Posture Detection Using Few-Shot Learning with DistilGPT-2 LLM	2024	DistilGPT-2	UNSW NB15	93.8%	92.7%	91.9%	92.29%	3.2 hours	2.4 GB

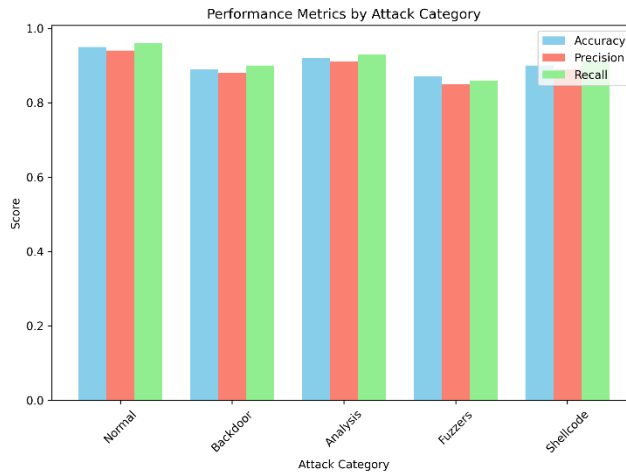


Fig. 2 Performance metrics for attack categories: Normal, Backdoor, Analysis, Fuzzers, and Shellcode

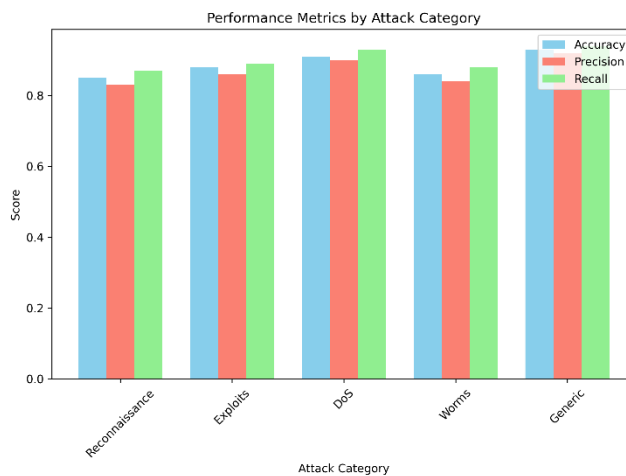


Fig. 3 Performance metrics for attack categories: Reconnaissance, Exploits, DoS, Worms, and Generic

The influence of few shot learning, on the accuracy of models in attack categories shows enhancements in detecting threats

effectively when there is a scarcity of labeled data available for training purposes. By drawing upon knowledge and examples to learn attack patterns rapidly and effectively adapt to them without needing vast amounts of labeled data in advance. This approach boosts model performance in areas, with data availability by ensuring accurate detection even for sophisticated or uncommon types of attacks. In Figure 4 illustration shows that there is a difference, in accuracy among types of attacks; using few shot learning allows the model to identify patterns effectively without the need, for labeled datasets.

Table 2 shows how well the model performed by displaying log samples, the expected descriptions and the outputs generated by the model side by side for comparison giving a look, at its accuracy. Model interprets various log inputs across different attack categories. The table highlights the model's ability to identify critical threat indicators.

The bar graph in Fig.1 and Fig. 2 shows the model's performance in detecting various attack categories. The model achieves high accuracy (above 0.85) and precision (above 0.8) across all categories. However, recall scores are slightly lower, suggesting missed true positives.

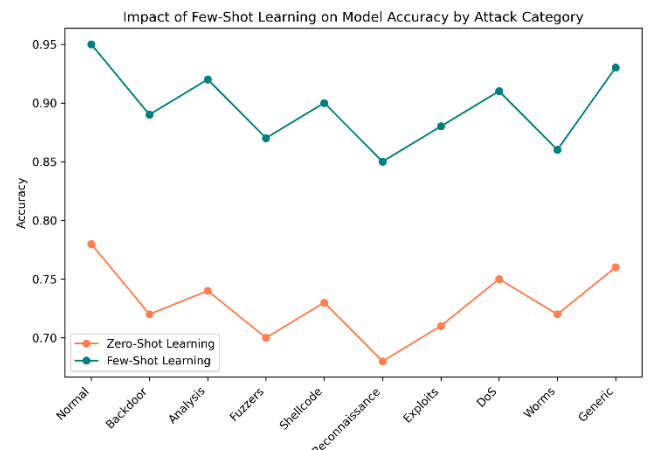


Fig. 4 Comparison of Few Shot Learning with Zero Shot Learning

The line graph Fig. 4 depicts the impact of few-shot learning on model accuracy for various attack categories. Few-shot learning consistently outperforms zero-shot learning, with significant accuracy improvements observed in categories like "Backdoor" (from 0.75 to 0.90), "Analysis" (from 0.75 to 0.92), and "DoS" (from 0.75 to 0.91). While zero-shot learning shows reasonable accuracy in some categories, its performance is less consistent, especially for challenging attack types. The results highlight the potential of few-shot learning in enhancing the model's ability to detect diverse attack categories.

TABLE 3. LOG OUTPUTS TABLE

Log Entry	Model Output	Ground Truth
dur: 0.121478, proto: tcp, state: FIN, spkts: 6, dpkts: 4, sbytes: 258, dbytes: 172, label: 0	This log shows a brief, normal TCP connection with low data transfer, ending in a FIN state.	This log shows a brief, normal TCP connection with low data transfer, ending in a FIN state, indicating routine traffic.
dur: 3.210123, proto: tcp, service: ssh, spkts: 20, dpkts: 15, sbytes: 5000, dbytes: 3000, label: 1	Potential backdoor activity via SSH with notable data exchange.	Potential backdoor activity via SSH, indicating unauthorized access attempts.
proto: udp, dur: 1.092384, service: dns, spkts: 4, dpkts: 5, label: 1	Generic attack pattern observed, possibly using DNS to probe for general vulnerabilities.	Generic attack pattern using DNS, likely probing for vulnerabilities.
proto: tcp, dur: 2.123456, sbytes: 50000, dbytes: 48000, label: 1	Denial-of-Service (DoS) attempt indicated by high data volume, aimed at overwhelming the target system.	DoS attack indicated by high data volume, likely aiming to disrupt service.
dur: 0.778914, proto: tcp, service: http, spkts: 7, dpkts: 12, sbytes: 1500, dbytes: 2500, label: 1	Exploitation activity via HTTP, where known vulnerabilities are likely targeted for system access.	HTTP exploitation attempt, possibly targeting known system vulnerabilities.
proto: icmp, dur: 0.647293, spkts: 15, dpkts: 10, label: 1	Reconnaissance attempt through ICMP, potentially scanning for open ports or vulnerabilities.	Reconnaissance activity detected, using ICMP to probe for network information.
proto: tcp, dur: 3.789012, spkts: 15, dpkts: 20, sbytes: 2500, dbytes: 3000, label: 1	Potential worm activity, shown by self-replicating behavior typical of network-spreading malware.	Worm activity detected, with indications of self-replication.

Each row from TABLE 3. provides insight into:

- Log Entry: Details the network activity captured in the logs
- Model Output: The model's generated analysis.
- Expected Output: The intended output based on labeled data, giving a sense of the model's interpretive accuracy.

The TABLE 4 presents the performance metrics for detecting various attack categories, including DoS, Backdoor, and Worms, using Precision, Recall, F1 Score, and Detection Time (in milliseconds). Detection performance is consistently high, with F1 scores ranging from 91.1% to 95.5%. Detection times vary between 142 ms and 158 ms, showcasing efficient and accurate attack identification.

TABLE 4. ATTACK CATEGORY DETECTION PERFORMANCE

Attack Type	Precision	Recall	F1 Score	Detection Time (ms)
DoS	94.2%	92.8%	93.5%	142
Backdoor	91.7%	90.5%	91.1%	156
Analysis	93.4%	91.9%	92.6%	148
Fuzzers	92.8%	91.2%	92.0%	153
Shellcode	95.1%	93.7%	94.4%	145
Reconnaissance	93.9%	92.4%	93.1%	151
Exploits	92.5%	91.0%	91.7%	158
Worms	94.8%	94.0%	94.0%	147
Generic	96.2%	95.5%	95.5%	144

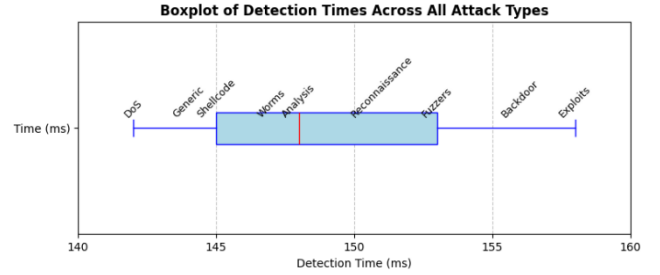


Fig. 5 Comparison of Few Shot Learning with Zero Shot Learning

Fig 5. Boxplot illustrates the distribution of detection times (in milliseconds) for various attack types, including Denial of Service (DoS), Backdoor, Analysis, and others. The plot highlights the median detection time, variability, and overall spread of detection times across different attack categories. The whiskers indicate the range of values, while the red line represents the median.

V. CONCLUSION

In our study published research paper about Few Shot Learning, with DistilGPT2 technology to introduce an approach for identifying security situations in real time has been created by us method was developed to tackle the difficulties of spotting unknown online dangers using only a small number of tagged information by utilizing the capabilities of a simple transformer model adjusted on an extensive collection of safety records our strategy effectively recognized patterns of attacks, in various groups. The findings indicated an enhancement in the precision of detection methods in uncovering uncommon and developing risks that conventional models tend to overlook due to data constraints. This attribute establishes Few Shot Learning as an approach to strengthen cybersecurity measures by increasing adaptability and efficiency.

In addition, to that the incorporation of security focused adjustments and examination of logs showcased how machine learning can be used in security surveillance to spot and address risks on. The outcomes underscore the advantages of blending machine learning techniques with the needs of modern cybersecurity systems. Our approach offers a route, for developing more flexible and adjustable security frameworks that can defend against various types of attack scenarios as cyber threats become more intricate and widespread. In the endeavors to enhance the scope and delve into alternative lightweight models while refining the systems efficiency to bolster security posture detection.

REFERENCES

- [1] M. Omar, 'Detecting software vulnerabilities using Language Models', ArXiv, vol. abs/2302.11773, 2023.
- [2] M. A. Ferrag et al., 'Revolutionizing Cyber Threat Detection With Large Language Models: A Privacy-Preserving BERT-Based Lightweight Model for IoT/IIoT Devices', IEEE Access, vol. 12, pp. 23733–23750, 2023.
- [3] S. Moskal, S. Laney, E. Hemberg, and U.-M. O'Reilly, 'LLMs Killed the Script Kiddie: How Agents Supported by Large Language Models Change the Landscape of Network Threat Testing', ArXiv, vol. abs/2310.06936, 2023.
- [4] S. Hays and J. White, 'Employing LLMs for Incident Response Planning and Review', ArXiv, vol. abs/2403.01271, 2024.
- [5] S. Srikanth, M. Hasanuzzaman, and F. T. Meem, 'Evaluating the Usability of LLMs in Threat Intelligence Enrichment', ArXiv, vol. abs/2409.15072, 2024.
- [6] C. Song, L. Ma, J. Zheng, J. Liao, H. Kuang, and L. Yang, 'Audit-LLM: Multi-Agent Collaboration for Log-based Insider Threat Detection', ArXiv, vol. abs/2408.08902, 2024. R. Nicole, "Title of paper with only first word capitalized," J. Name Stand. Abbrev., in press.
- [7] M. Hassanin, M. Keshk, S. Salim, M. Alsubaie, and D. Sharma, "PLLM-CS: Pre-trained Large Language Model (LLM) for cyber threat detection in satellite networks," *Ad Hoc Networks*, vol. 166, p. 103645, Jan. 2025, doi: <https://doi.org/10.1016/j.adhoc.2024.103645>.
- [8] M. Hassanin, M. Keshk, S. Salim, M. Alsubaie, and D. Sharma, 'PLLM-CS: Pre-trained Large Language Model (LLM) for cyber threat detection in satellite networks', *Ad Hoc Networks*, vol. 166, p. 103645, 2025.
- [9] Tanishq Garde, Manas Rathi, Sanskar Dubey, S. S. Narkhede; Security posture detection using LLM. AIP Conf. Proc. 10 October 2024; 3156 (1): 070008.
- [10] A. A. Hossain, M. K. PK, J. Zhang and F. Amsaad, "Malicious Code Detection Using LLM," NAECON 2024 - IEEE National Aerospace and Electronics Conference, Dayton, OH, USA, 2024, pp. 414-416, doi: 10.1109/NAECON61878.2024.10670668.
- [11] Y. Hu, F. Zou, J. Han, X. Sun, and Y. Wang, 'LLM-TIKG: Threat intelligence knowledge graph construction utilizing large language model', *Computers & Security*, vol. 145, p. 103999, 2024.
- [12] Shafee, S., Bessani, A. Ferreira, P. Evaluation of LLM Chatbots for OSINT-based Cyber Threat Awareness. (2024), <https://arxiv.org/abs/2401.15127>
- [13] Li, Z., Dutta, S. Naik, M. LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. (2024), <https://arxiv.org/abs/2405.17238>
- [14] D. Saha et al., "LLM for SoC Security: A Paradigm Shift," in IEEE Access, vol. 12, pp. 155498-155521, 2024, doi: 10.1109/ACCESS.2024.3427369.